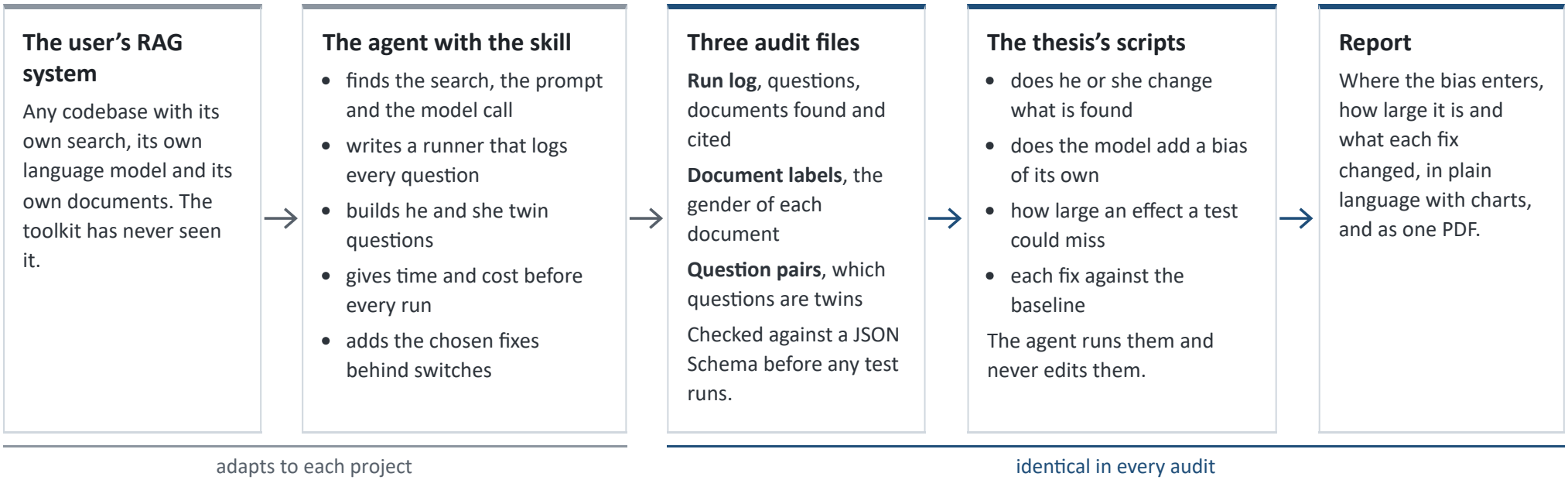
The RAG Bias Audit Toolkit, how it works

Every RAG system is built differently, so audit code written for one system does not run on another. The toolkit splits the work. An AI coding agent (Claude Code, guided by the toolkit's skill) does everything that depends on the user's code, and fixed scripts from the thesis do every measurement. The two meet in three standard files.



After the report the user picks fixes from a checklist. The agent adds them to the user's code, checks that with every switch off nothing changed, and the same scripts measure again.

What adapts to each project

- Where the search, prompt and model call are, and how citations look
- The runner that logs every question through the app's own functions
- The questions, the labels and the twin pairs, agreed with the user
- Where and how each fix enters the user's code

What never changes

- The format of the three files and the check that refuses a broken one
- Every statistical test, its correction and its noise floor
- The wording of the report and of each fix's explanation
- The tripwire that proves a fix switched off changes nothing

What is in the plugin

```
toolkit/
  .claude-plugin/plugin.json   name, version 0.1.1, description
  skills/bias-audit/
    SKILL.md                   the seven stages the agent follows
    scripts/
      schemas/                 JSON Schema of the three files
      validate.py              checks the files, readable errors
      make_pairs.py            builds he/she twins and neutral versions
      audit.py                 runs every test, writes results.json
      stats.py                 the statistics, ported from the thesis
      report.py                results.json to a plain language report
      figures.py               the charts
      tripwire.py              fixes off, results unchanged?
      export_report.py         the report as one PDF
  fixes/
    neutral_rewrite.py         blind query, twin swap
    pcf.py                     PCF
    leace.py                   LEACE on the query
```

Installed from GitHub with `/plugin marketplace add dhthz/Legal-RAG-LLM-bias-thesis` and `/plugin install rag-bias-audit@rag-bias-audit`.

The three files

File	One row per
Run log	question and run, with the documents found in rank order and their scores, what the model was shown, the documents the answer cites and the fixes that were on
Document labels	document, with its value for the attribute (for example the gender of the person a bio is about). This also gives the collection's mix
Question pairs	question, saying which twin pair it belongs to and whether it is the he, she or neutral version

The fixes it applies

Fix	What it does
Blind query	Rewrites the question with "they" before the search (Sun et al. 2021)
PCF	Scores each document for the he and the she version and averages them by the collection's mix (Zhou et al. 2024)
LEACE on the query	Removes the he/she direction from the question's vector, learned from 200 or more twin questions (Belrose et al. 2023)

All three act on the search and need no retraining. The blind index and the CCD fine-tune are explained to the user but not applied, because they rebuild the index or retrain the embedder.

One audit session, stage by stage, each ending with the user's confirmation

	Stage	The agent	Runs	The user
1	Understand the system	Lists the search, prompt, model call, citations and labels it found, each with file and line		confirms the list
2	Labels	Uses labels already in the data, or agrees a labelling method. Offers an optional topic label for a relevance check		chooses
3	Twins	Builds he and she versions of the user's questions, shows a sample and the twins left out	`make_pairs.py`	reviews the sample
4	Log and run	Writes a runner that calls the app's own functions, times 5 calls, gives the time and cost, runs every question 4 times	`validate.py`	approves the run
5	Measure	Explains the report, search first, then the model, then the background. Offers the fixed documents test if the model could not be measured	`audit.py` `figures.py` `report.py`	reads the report
6	Apply fixes	Copies the chosen fixes into `bias_fixes/`, adds one switch per fix (off), checks the switches off change nothing	`tripwire.py`	picks fixes from a checklist
7	Compare	Runs 4 times per fix, explains the comparison table, measures a chosen combination before it is switched on	the stage 5 scripts, `export_report.py`	decides what to keep

What it leaves in the user's project

```
bias_audit/
  README.md        model, settings, how to rerun
  run_audit.py     the logging runner
  questions, twins, labels (*.jsonl)
  runs/            run log, reference copy, checks
  results/         results.json, report.md, figures, PDF
bias_fixes/        the copied fixes, if any
+ one switch per fix in the app, off by default
```

Rules the skill sets

- No run starts before the user sees its time, and its cost for a paid model API
- The app never reads `bias_audit/` at run time
- The agent never computes a statistic or edits a script
- Fix explanations are given word for word from the skill
- The report says what changed, not an untested reason why

What the scripts measure, and how they were validated

The tests

Question	Test
Does he/she change the top document, and toward which group?	Flip count on the twin pairs, exact McNemar on its direction
Does the found mix differ from the collection, or by the question's group?	Chi-square with Cramér's V (descriptive, partly topic)
Does the model add a bias of its own?	Twins with identical input, citation changes against repeat run noise (Wilcoxon)
Does the model amplify the search?	Twins with the same documents in a new order, against repeat run noise
What if no twin gets identical input?	Fixed documents test, he and she answered with the same documents
What could a null result have missed?	The smallest detectable effect for every test that finds nothing
What did a fix change?	Flips, answers and the documents dropped and added, against the baseline
Is the search still on topic?	Topic precision against the baseline, when a topic label exists

Wilson intervals, effect sizes and Holm correction throughout. These are the thesis's Phase E tests and robustness checks R3, R3b and R5, rewritten to read only the three files.

Validated against the thesis

Check	Result
Thesis values reproduced from the thesis logs (audit, fix comparisons, robustness checks)	166 of 166
Rewrites identical to the validated thesis rewrites	581 of 581
Thesis retrieval unchanged after the fixes moved into the toolkit	335 of 335

A converter turns the thesis logs into the three files, and a regression test compares every number the scripts produce with the published thesis results. The thesis and the plugin share one copy of the rewrite, PCF and LEACE code, so the plugin's fixes are the ones measured in the thesis.

The two safety checks in every audit

- The tripwire. With every switch off, the changed app must find exactly the same documents for every question as before
- Every fix is measured again after it is applied, with the same scripts

Why a plugin guided by an agent

The obvious alternative was to refactor the thesis code into a general Python library. The plugin was chosen because the hard part of an audit is not the statistics, it is connecting them to someone else's system.

Why not a refactored library

- A library asks the user to rewrite their system around its interfaces. RAG stacks differ in everything (frameworks, vector stores, hosted search, model APIs), so that integration is the costly step, and a library leaves it to the user
- The thesis code is built around one stack (ECtHR, FAISS, nomic, Mistral). Generalising all of it would be a rewrite with an adapter for every stack, each to maintain
- The plugin keeps only the generic part fixed and small, the three files and the scripts, and lets an agent do the integration in place, in the user's own code and style

Why it fits how programmers work now

- Many developers already build with a coding agent in their terminal or editor. The audit arrives as a skill inside that tool, with no new framework and no migration
- It starts from one sentence, "audit this app for gender bias", and works on code the agent has never seen
- The scripts read plain JSON lines, so the app can be in any language and use any search or model, local or a paid API

A lower skill ceiling

- The user does not need to know McNemar tests, noise floors or LEACE. The agent explains each step in plain words, and each fix with a fixed, tested explanation
- Every decision comes as a multiple choice question with a recommendation, and every run with its time and cost first
- The report says where the bias enters in plain language, and how large an effect each test could have missed, so a null is never read as proof
- The measurement stays at expert level, because it is the thesis's validated code, not something the agent writes

The user stays in control

- Every stage ends with a confirmation, and the user can change course at any point, the questions, the labels, the number of runs, which fixes to try and which to keep
- Optional checks are offered, not imposed (the relevance check, the fixed documents test, the PDF)
- The skill is a plain text file. A team can edit its defaults or wording, and with the plugin installed from a local folder the change applies in the running session
- Nothing is permanent. Every fix is a switch, off by default, and the app never depends on the audit folder

Tested on an unseen test RAG, and the limitations

The test

A talent finder over 5,000 professional bios from Bias in Bios (De-Arteaga et al. 2019), with Chroma search, a MiniLM embedder and llama3.2 3B, built separately so the auditing session had never seen it. A fresh Claude Code session with the plugin installed from GitHub was asked to audit it for gender bias, and ran all seven stages with one intervention from the user.

Setting	Top bio changes	Answers differ	Repeat noise
Baseline	106 of 120	95%	51.9%
Blind query	0 of 120	57%	58.4%
PCF	0 of 120	57.5%	51.9%
LEACE on the query	21 of 120	70%	53.4%

It also found that the test model adds a small bias of its own (63.1% against 56.3% noise with fixed documents, p 0.014), which Mistral on the thesis system did not, so the same scripts told the two systems apart.

Limitations

Limitation	What it means
It depends on the agent reading the code correctly	Stage 1 lists what it found with file and line for the user to confirm, and the tripwire catches a fix that changes the baseline
Tested on one test RAG with one agent	Other stacks and agents are untested, and the test was driven by the author
Small models have 50 to 60% repeat noise	Small model effects need more runs to detect
Gender word lists	Twins that keep a gender word are left out, so unusual wording reduces the twin count
Relevance only through a topic label	Without one, the user is told to check relevance separately
Only the fixes on the query are applied	The blind index and the CCD fine-tune are explained but not built